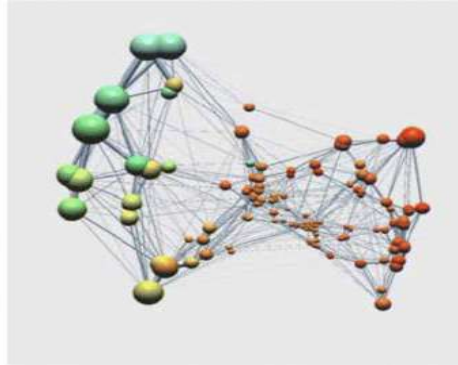
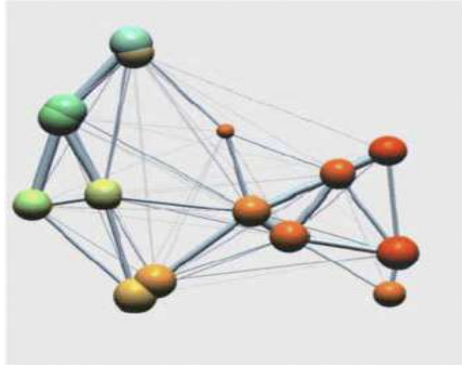


Analyzing Graph Partitioning Algorithms on the MBTA Subway Transit Network



By:

Aryan Meena

Nida Munawar

Pranjal Chandravanshi

Main Goal:

Apply and evaluate graph partitioning algorithms to analyze, understand and improve the MBTA subway system structure.

Algorithms Explored:

- **Louvain Method** – community detection via modularity maximization
- **Spectral Clustering** – uses graph Laplacians and eigenvalues
- **METIS** – partitions while minimizing edge cuts and balancing nodes

Why Study the MBTA Subway Network?

- **Complex, real-world transit system with over 100 stations**
- **Critical for Boston's daily commuters and city connectivity**
- **Offers an ideal case for applying graph theory and partitioning algorithms**

MBTA Subway Graph

Dataset Overview

Data sourced from MBTA GTFS feed - Included subway services only - related stops, trips, routes, and stop times

The subway network was modeled as a graph with **106 stations as nodes** and **113 direct connections as edges**.

Each edge represents a pair of stations that are directly connected in at least one subway trip.

Assigned **weights to the edges** based on how frequently those station pairs appeared consecutively in the data.

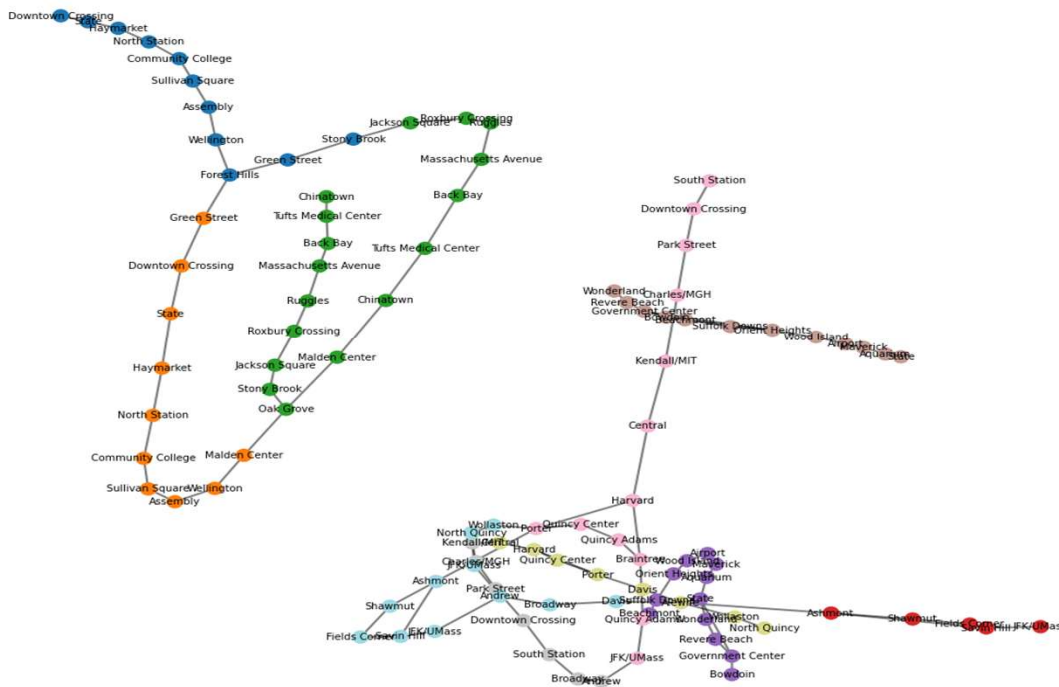
MBTA Subway Network (Before Louvain Community Detection)



Louvain Partitioning Algorithm for Community Detection

A fast, community detection method that groups nodes by maximizing **modularity**, meaning it finds clusters with dense internal connections and sparse external ones.

MBTA Subway Network with Louvain Communities and Stop Names



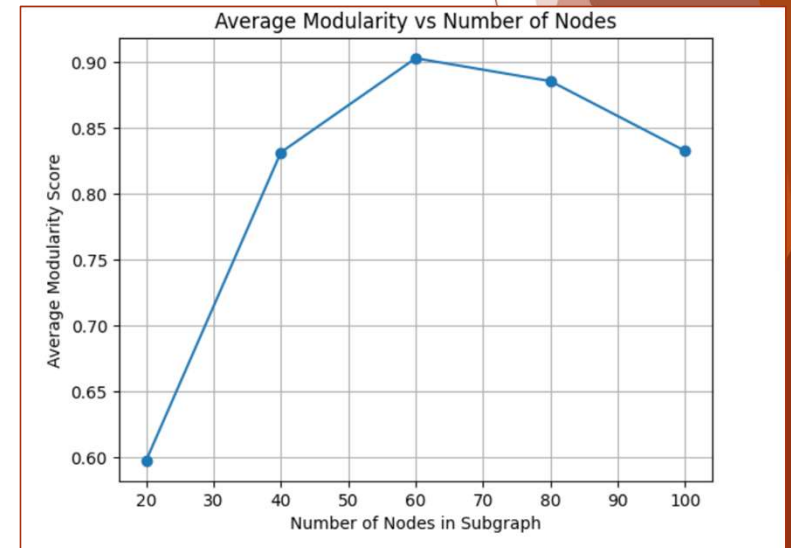
- Grouped subway stations into **10 communities**
- Maximized **modularity** to find tightly connected groups
- Revealed hidden structure within the subway network
- Louvain partitioning helps **visualize hidden structure** in complex transit networks
- Supports **smarter planning, congestion management, and route optimization**

Modularity Analysis of Louvain Algorithm

- Modularity evaluates how well the network is divided into distinct communities.
- Achieved a high **modularity score of 0.8113**
- Indicates well-separated, meaningful station clusters

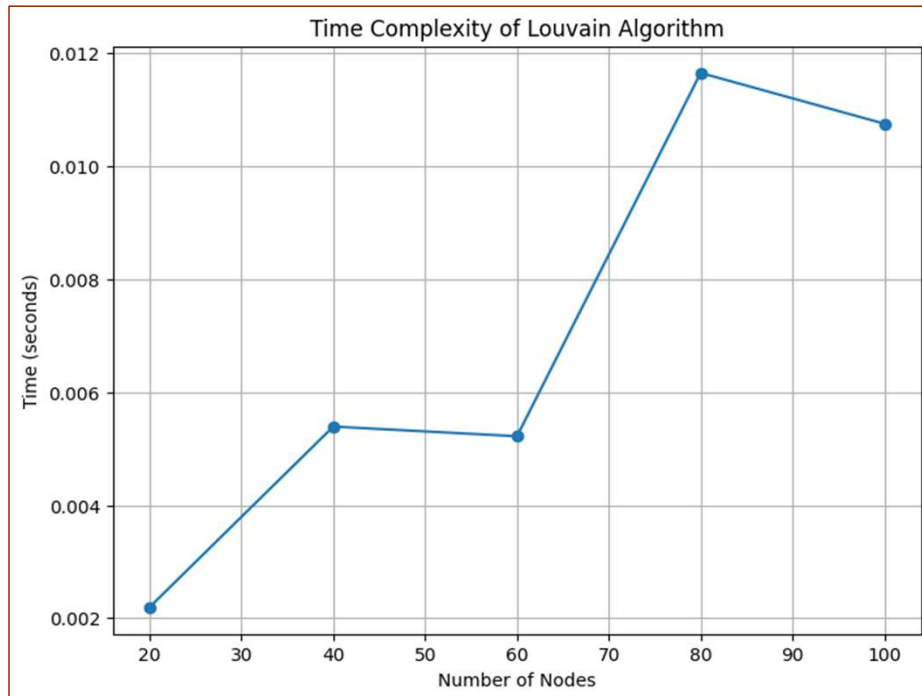
- To test consistency, Louvain applied to subgraphs of increasing sizes (20 to 100 nodes)
- Each size tested multiple times for average modularity.
- Modularity improves with size peaking at 60-80 nodes
- Louvain is scalable and effective for large networks

Average Modularity Scores by Subgraph Size:	
Number of Nodes	Average Modularity
20	0.5972
40	0.8314
60	0.9029
80	0.8855
100	0.8324



Louvain Algorithm – Time & Space Complexity

- Louvain tested on subgraphs of 20–100 nodes
- Time taken increased with size, showing $O(n \log n)$ trend



- Memory usage remained low and stable during execution
- Graph creation and community detection added minimal overhead

Memory usage before graph creation: 88.46 MB

Memory usage after graph creation: 88.48 MB

Memory usage after Louvain: 88.57 MB

Evaluating Louvain Community Structure

Edge Weights Between Communities

- ▶ Total edge weight between partitions: 8481
- ▶ A lower number here is good—it means most connections are **within communities** rather than between them
- ▶ Confirms effective separation of station groups

Node Partition Weight Balance

- All 10 communities had **equal internal edge weight**
Partition weight differences: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- Indicates **perfect balance** and **fair distribution** across the network

Partitioning using METIS

- **METIS** partitions a graph into k roughly equal parts while minimizing the number of edges *cut* between parts.
- **METIS uses a multilevel scheme:**
 1. **Coarsening** – collapse nodes/edges to form successively smaller graphs.
 2. **Initial partitioning** – split the tiny graph into k parts.
 3. **Uncoarsening & refinement** – project the partition back to the original graph, locally improving cut size.

Partition Sizes & Balance

Pre-processing:

We built an undirected graph G where

$V = 265$ stations (from stations.csv)

$E = 541$ edges (connections.csv), one per consecutive stop in any trip

No edge weights were assigned in this first pass—so METIS treats each link equally.

```
... print( stations, len(stations), connections, len(edges_undirected))
Stations: 265 Connections: 541
```

We chose $k = 3$

Post Processing:

```
... print('Balance ratio  $\sigma = \max |V_i| / [n/k] = 1.07$ ')
• Partition 1: 86 nodes
• Partition 2: 85 nodes
• Partition 3: 94 nodes
• Balance ratio  $\sigma = \max |V_i| / [n/k] = 94/88 \approx 1.07$ 
```

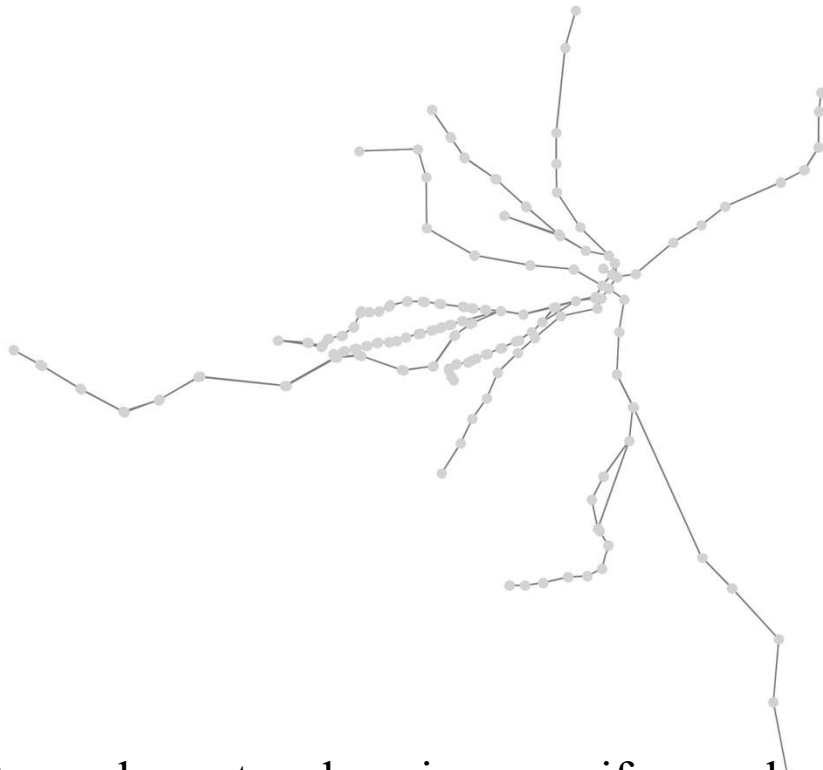
```
...
```

Graph: 265 stations, 276 edges; partitioning into $k=3$ parts

Comparisons

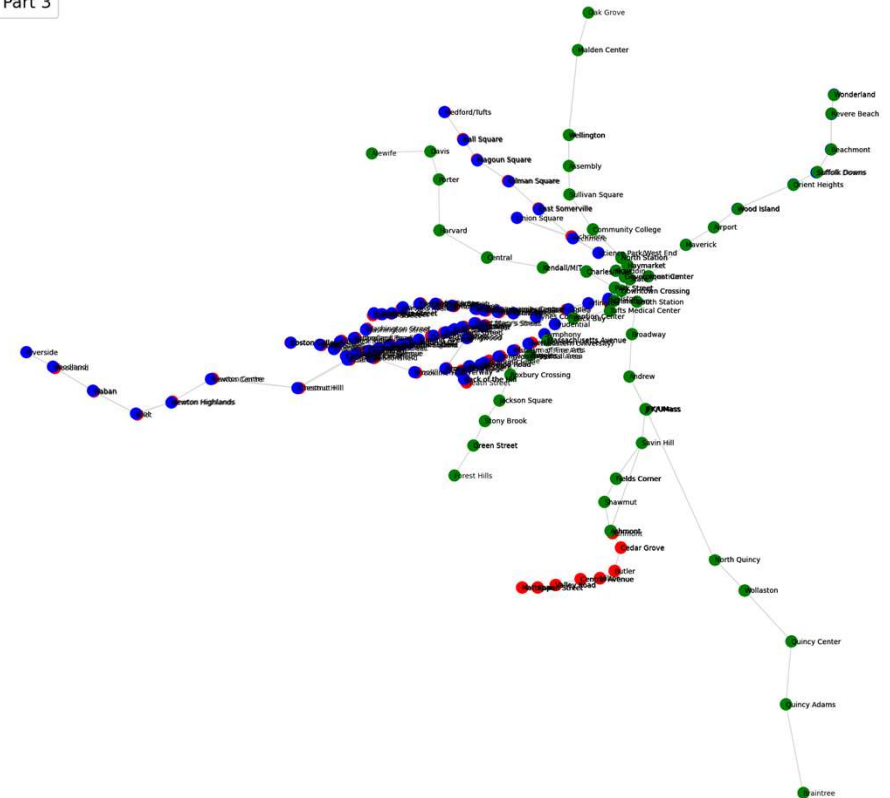
Before METIS

Left (“Before METIS”)



After METIS (k=3) — Labeled

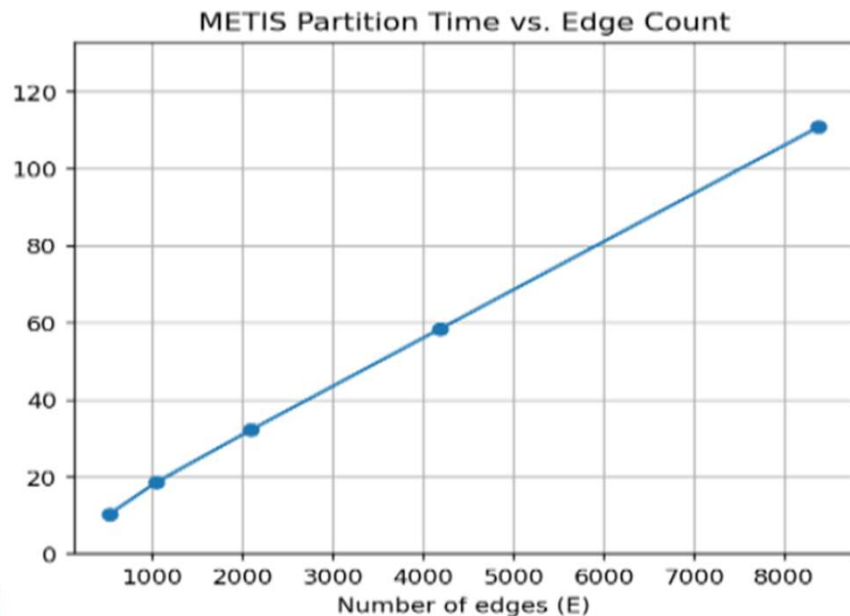
Right (“After METIS”)



- Raw subway topology in one uniform color.
- All the branches radiating from the core, but no structural grouping is highlighted.

Scalability and Performance

Linear-Time Behavior: METIS's three phases (coarsening, initial partitioning, refinement) **each touch every edge** only a **constant number of times**, so the runtime grows roughly as **$O(E)$** for sparse graphs.



Practical Speed:

- On our MBTA network of 276 edges.
- METIS completes partitioning in just milliseconds.
- Suitability for even much larger transit systems.
- **Maximum possible edges** (in a fully connected graph) on 265 nodes would be
- **$265 \cdot (265 - 1) / 2 \approx 35,028$**
- **Actual edges in our network are only 541, which is less than 2% of that maximum.**
- Equivalently, the average degree is $E/V = 2 \cdot 541 / 265 \approx 4.08$
- **meaning each station connects directly to only about four others on average.**

Spectral Clustering

- ▶ Treat transit network as a graph of stations (nodes) and track connections (edges).
- ▶ Build Laplacian $L=D-A$; eigenvectors of smallest non-zero eigenvalues encode community structure.
- ▶ Form $n \times k$ spectral embedding with the next k eigenvectors; run k-means to obtain clusters.

$$A = [A_{ij}], \quad A_{ij} = \begin{cases} 1 & \text{if station } i \text{ is directly connected to } j, \\ 0 & \text{otherwise;} \end{cases}$$

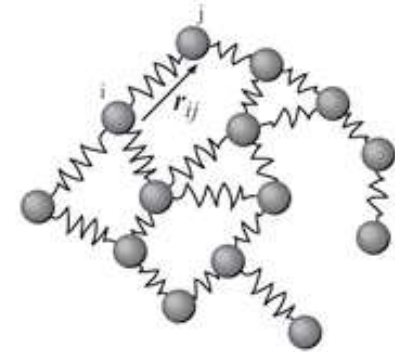
$$D = \text{diag}(k_1, k_2, \dots, k_n), \quad k_i = \sum_{j=1}^n A_{ij} \quad (\text{degree of node } i).$$

Then

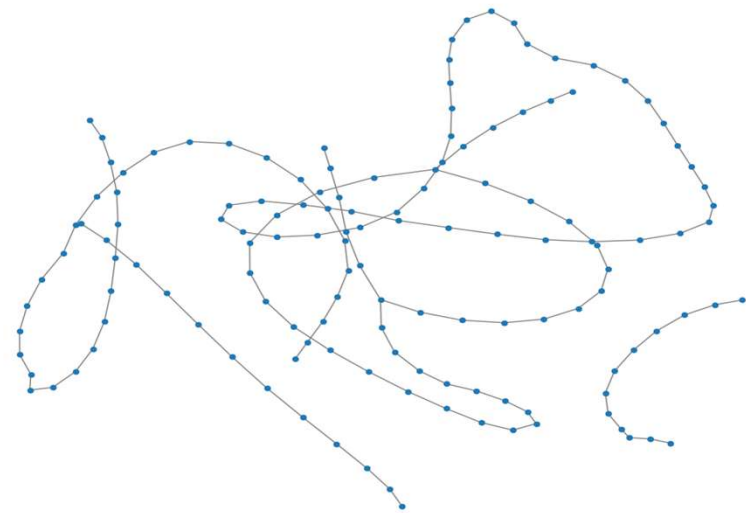
$$L = D - A = \begin{bmatrix} k_1 & & \\ & \ddots & \\ & & k_n \end{bmatrix} - \begin{bmatrix} A_{11} & \dots & A_{1n} \\ \vdots & \ddots & \vdots \\ A_{n1} & \dots & A_{nn} \end{bmatrix}$$

Spectral Embedding & Clustering

- ▶ Compute $L=D-A$.
- ▶ Solve $Lv=\lambda v \rightarrow$ keep eigenvectors $v_1...v_5$ (skip v_0).
- ▶ Matrix $X=[v_1...v_5]$ (5-D coordinates).
- ▶ k-means, $k = 5 \rightarrow$ clusters $\approx$ Red, Orange, Blue, Green, Mattapan.



MBTA Subway Graph Before Clustering (Spring Layout)



$$Lv_\ell = \lambda_\ell v_\ell, \quad 0 = \lambda_0 \leq \lambda_1 \leq \dots \leq \lambda_{n-1}.$$

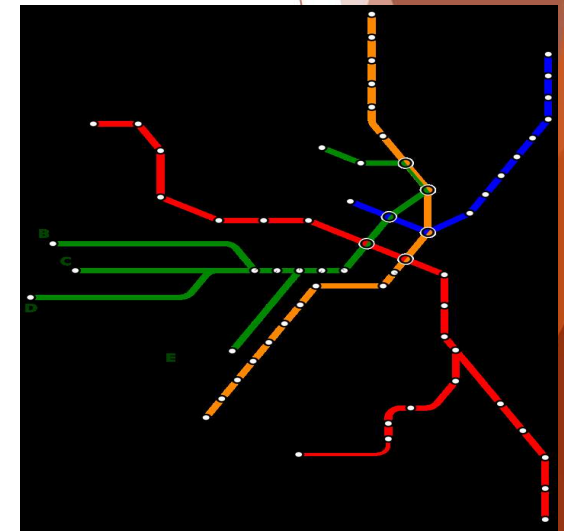
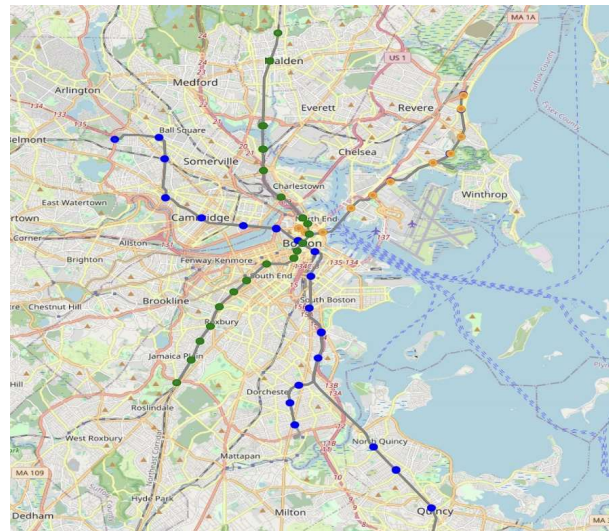
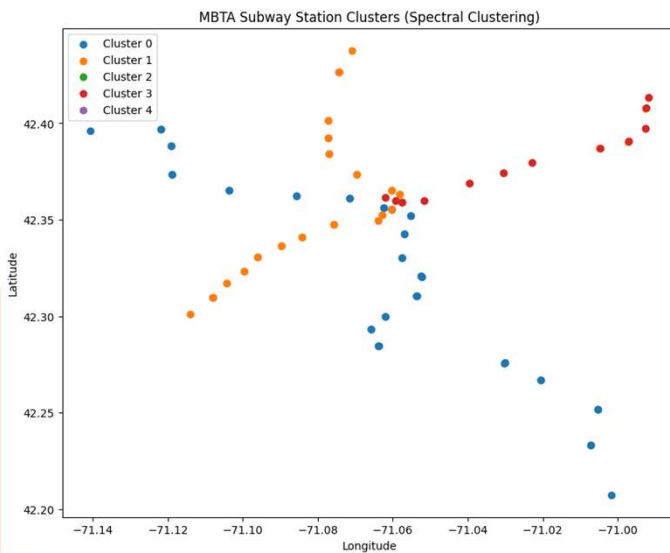
Discard v_0 (the all-ones vector) and form

$$X = [v_1 \ v_2 \ \dots \ v_k] \in \mathbb{R}^{n \times k}.$$

Row i of X is the k -dimensional coordinate of station i fed to k -means.

Cluster Visualizations

- ▶ Scatter: stations plotted by lat/long, coloured by cluster (Fig.1).
- ▶ Folium map: interactive HTML with coloured markers (Fig.2).
- ▶ Matches official MBTA line map (Fig. 3) with one cross-cut at Ashmont-Mattapan.



Spectral Clustering Metrics & Performance

--- Spectral Clustering Metrics ---

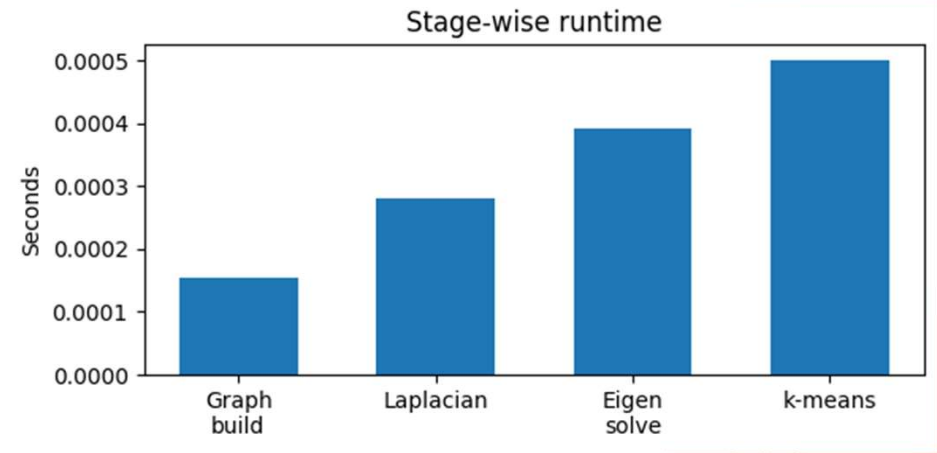
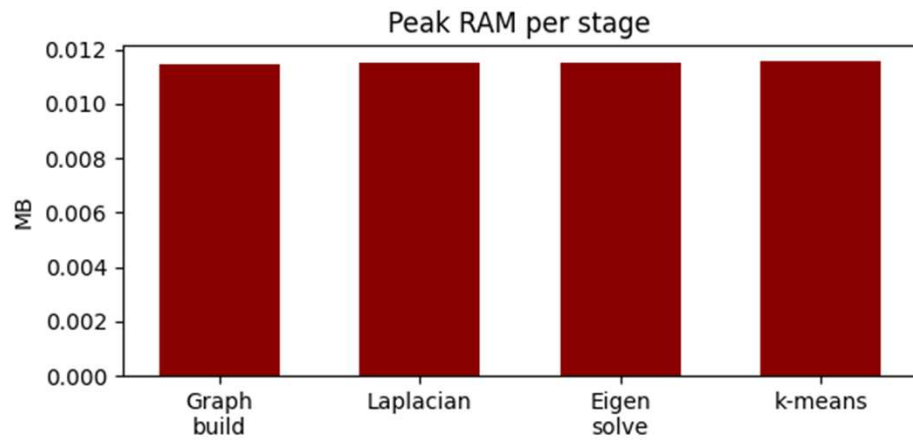
Number of Clusters : 5
Modularity : 0.7239
Edge Cut : 1
Average Conductance : 0.0125
Node Partition Balance : 38

Modularity | **0.72** | Strong intra-cluster density |
| Edge-cut | **1** | Only Mattapan-Ashmont link crosses clusters |
| Conductance | **0.0125** | Very clean separation |
| Node-balance | **38** | Green cluster >> Mattapan size

Stage	Big-O	MBTA numbers	Concrete cost
Filtering + I/O	$O(N)$	53 K rows	<0.1 s
Edge list build	$O(T \cdot s)$	$T=212, s=30 \rightarrow 6\,360$ pairs	<1 ms
Graph construction	$O(E)$	$E=62$ edges	62 inserts
Adjacency / Laplacian	$O(n^2)$	$n=63 \rightarrow 3\,969$ entries	≈ 0.004 MB, $\ll 1$ ms
Eigen solve	$O(n^3)$	$63^3=250\,047$ flops	0.0004 s, 0.04 MB
k-means (k=5)	$O(n \cdot k \cdot i)$	$63 \times 5 \times 10 = 3\,150$ dists	<1 ms
Visualisation	$O(n)$	63 points / markers	negligible

- Eigen-solve **0.001 s**, peak RAM **0.04 MB**.
- Entire pipeline sub-millisecond; scales to larger graphs

Peak RAM and Stage-wise runtime Plots



Comparative Insights

Criterion	Spectral	Louvain	METIS
Modularity	0.72	0.81	0.66
Edge-cut	1	8 481	3
Conductance	0.012	0.015	0.025
Auto-select k	No	Yes	No
Runtime	0.0004 s	0.002 s	0.005 s
Best for	Line recovery, spectral insight	Fine communities, high modularity	Balanced partitions, large graphs

Conclusion & Future Work

- ▶ Spectral recovers whole-line communities with minimal cuts and trivial compute cost.
 - ▶ Louvain delivers highest modularity and branch-level detail.
 - ▶ METIS provides size-balanced partitions useful for load balancing.
 - ▶ Together, they give a multi-resolution view of MBTA connectivity.
-
- ▶ Weight edges with ridership / travel time.
 - ▶ Overlapping or fuzzy communities for transfer stations.
 - ▶ Peak vs off-peak temporal graphs.
 - ▶ Hybrid pipelines: METIS coarse → Louvain fine.

References

- ▶ Blondel et al., 2008 - Louvain.
- ▶ Von Luxburg, 2007 - Spectral clustering tutorial.
- ▶ Karypis & Kumar, 1998 - METIS.
- ▶ Rosvall & Bergstrom, 2008 - Infomap.
- ▶ Google Developers - GTFS static spec.
- ▶ MBTA GTFS feed (2024-03-19).
- ▶ Hagberg et al., 2008 - NetworkX.
- ▶ Scikit-learn docs - SpectralClustering, KMeans.
- ▶ python-louvain pkg.
- ▶ Folium mapping library.

THANKYOU